

Ans 25

1) Given C code:  $a = \$t0, b = \$t1, \$c = \$s0,$   
 $d = \$s1, e = \$s2$

$a = b + c;$

$d = a - e;$

MIPS Assembly Translation

$\text{add } \$t0, \$t1, \$s0$

$\text{Sum } \$s1, \$t0, \$s2$

Machine code (Binary)

i)  $\text{add } \$t0, \$t1, \$s0$

R-type format:

| OP     | rs    | rt    | rd    | shamt | funct  |
|--------|-------|-------|-------|-------|--------|
| 000000 | 01001 | 10000 | 01000 | 00000 | 100000 |

~~ii) Sub~~

$\$t0 = \$8$

$\$s0 = \$16$

$PC1 =$

$\$s1 = \text{address of } \$s0$

$\$s1 - 2 = 0$

$\$s1 - PC1 =$

$\Gamma =$

ii) `sub $s1, $t0, $s2`

R-type format:

| op     | rs    | rt    | rd    | shamt | funct  |
|--------|-------|-------|-------|-------|--------|
| 000000 | 01000 | 10010 | 10001 | 00000 | 100010 |

1) If both variables are stored at the same address, writing to 'a' overwrites the value of e.  
So, after the first instruction ( $a = b + c$ ), the value of 'e' also becomes equal to 'a'.

Thus, when the second instruction `eq` executes ( $d = a - e$ ), it becomes:

$$d = a - a$$

$$d = 0$$

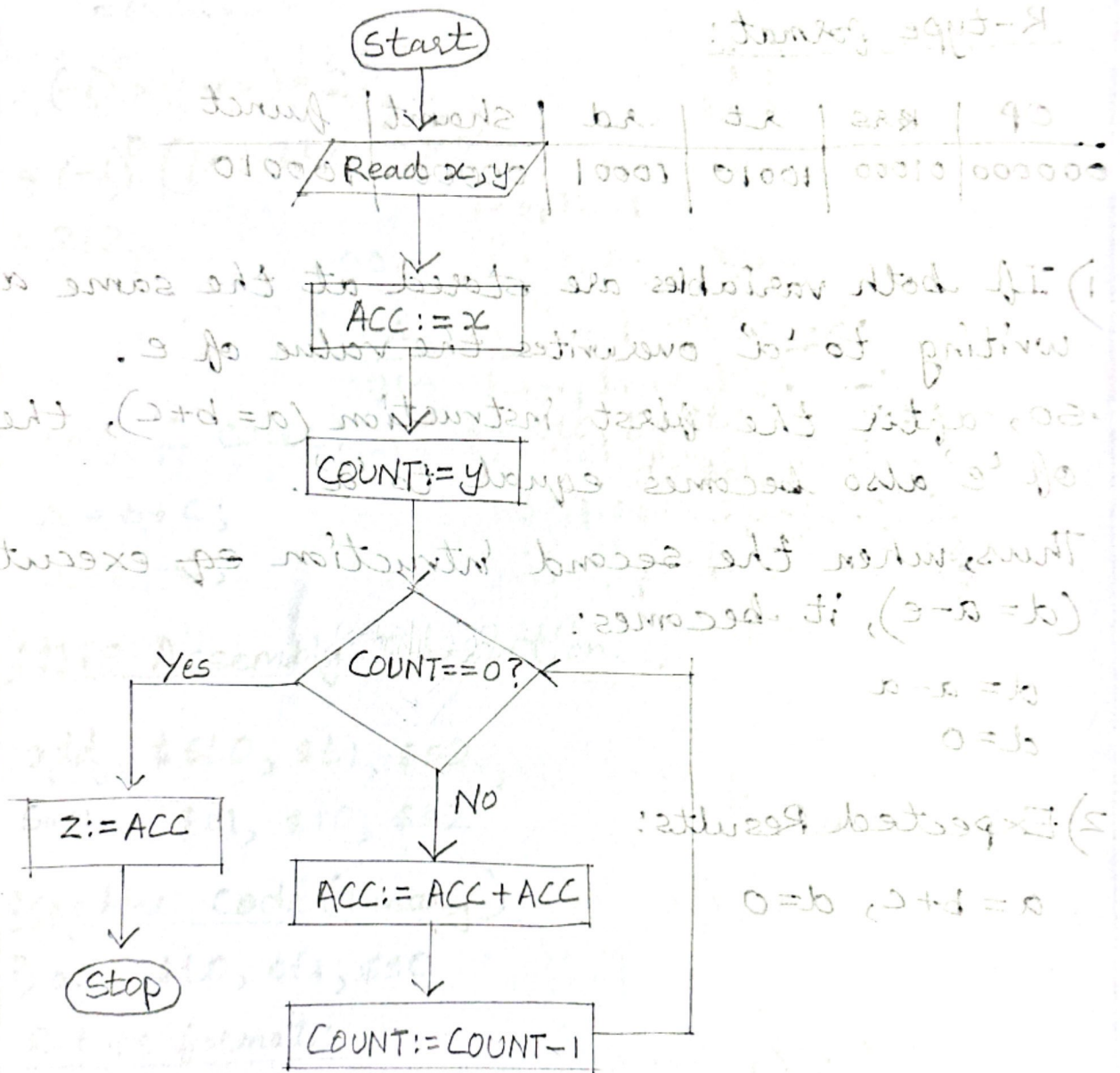
2) Expected Results:

$$a = b + c, d = 0$$



2)

Flow chart (students incorrect design)



1) The result of  $z = x * y$  may be incorrect because the student stored  $x$  in the same location used as the ~~runner~~ running sum (ACC), the original multiplicand  $x$  is overwritten on the first addition. Subsequent additions therefore use the updated accumulator value instead of the original  $x$ . That

changes each addition operand from the constant  $x$  to a changing value, so the loop does not compute  $z = 0 + x + x + \dots$  ( $y$  times), and the final  $z$  is wrong.

## 2) Step-by-step sequence of additions;

Assume  $ACC$  initially  $:= x := 4$ , loop runs 3 times and each iteration does  $ACC := ACC + ACC$  because  $x$  has been overwritten:

- \* Initial:  $ACC = 4$
- \* After 1st iteration:  $ACC = 4 + 4 = 8$
- \* After 2nd iteration:  $ACC = 8 + 8 = 16$
- \* After 3rd iteration:  $ACC = 16 + 16 = 32$

Final  $z = ACC = 32$  (incorrect),

Correct product  $x * y = 4 * 3 = 12$ , so the faulty reuse produces the wrong answer.

## 3) Use a 1-word temporary register

- \* Copy  $x$  into a small temp register  $TMP$  ( $TMP := x$ )
- \* Initialize  $ACC := 0$ .
- \* Loop  $y$  times:  $ACC := ACC + TMP$
- \* This uses exactly one extra small register and guarantees correctness with minimal hardware.